# STANDARDS AND INFORMATION DOCUMENTS

**AES31-3-2008**
**(reaffirmed 2019)**


## AES standard on Network & file transfer of audio — Audio-file transfer and exchange Part 3: Simple project exchange



**AUDIO ENGINEERING SOCIETY, INC.**
**60 East 42nd Street, New York, New York 10165, US.**

**STANDARDS**

The AES Standards Committee is the organization responsible for the standards program of the Audio Engineering Society. It publishes technical standards, information documents and technical reports. Working groups and task groups with a fully international membership are engaged in writing standards covering fields that include topics of specific relevance to professional audio. Membership of any AES standards working group is open to all individuals who are materially and directly affected by the documents that may be issued under the scope of that working group.

Complete information, including working group scopes and project status is available at http://www.aes.org/standards. Enquiries may be addressed to standards@aes.org

The AES Standards Committee is supported in part by those listed below who, as Standards Sustainers, make significant financial contribution to its operation.

Standards Sustainer Gold

NETFLIX

Focusrite

Standards Sustainer Silver

LECTROSONICS

21ST CENTURY FOX

Audio precision

WAVES

SHURE

fulcrum ACOUSTIC

ROSS LIVING LIVE!

Eventide

PreSonus

DOLBY

THE TELOS ALLIANCE

SENNHEISER

BOSE

ADAMSON

MUSIC TRIBE

Standards Sustainer

NHK

NTi AUDIO

MQA

audio-technica

LAWO

CLAIR

WEISS

AEA

UNSOUND

THAT Corporation

NBCUniversal

Control 4

WorldCast Systems

Fraunhofer IIS

PRO AUDIO TECHNOLOGY

MERGING AUDIO FOR THE NETWORKING AGE

RAVENNA

steinberg

ARCHWAVE audio networking & streaming technologies

ATTERO TECH

HARMAN A SAMSUNG COMPANY

KLIPPEL

NEUTRIK

dbx-tv

XPERI

This list is current as of 2019/1/30

**AES31-3-2008**
**(reaffirmed 2019)**

# AES standard for network and file transfer of audio - Audio-file transfer and exchange - Part 3: Simple project interchange

Published by
**Audio Engineering Society, Inc.**


**Abstract**

This standard provides a convention for expressing edit data in text form in a manner that enables simple and accurate computer parsing while retaining human readability. It also describes a method for expressing time-code information in character notation and simple automation for stereo & surround panning and audio gain.



**Audio Engineering Society Inc., 551 Fifth Avenue, Room 1225, New York, NY 10176, US.**
www.aes.org/standards standards@aes.org

2019-04-23 printing

## Contents

## Foreword

[This foreword is not a part of *AES standard for network and file transfer of audio — Audio-file transfer and exchange — Part 3: Simple project interchange,* AES31-3-1999.]

In 1997-03, the SC-06-01 Working Group on Audio-File Transfer and Exchange of the SC-06 Subcommittee on Network and File Transfer of Audio replaced SC-02-08 to consider the topic of interchange of audio material in the form of computer file data.

This document is part of a proposed multi-part standard for a simple format for audio project interchange defined in project AES-X67. It describes a generalized means to interchange edit data in the form of human-readable text. It describes a means to communicate timing information to sample accuracy in the form of a text string. It describes a technique for communicating edit information between different types of computer platform using plain text.

The remaining projects at the time of issue of this printing, and their intended parts, are:

Project AES-X69: AES31-1 *Media for Interchange;*
Project AES-X66: AES31-2 *Audio File Format;*
Project AES-X68: AES31-4 *Object Oriented Project Interchange.*

Following the presentation of an earlier version of AES31-3 at the AES 105th Convention in San Francisco, CA, US, the draft was substantially revised by Task Group SC-06-01-C at Oxford, England, UK, in 1999-02. It was completed and issued with a call for comment following the working group meeting in conjunction with the AES 106th Convention in Munich, Germany, 1999-05.

At the time of public approval of the standard, the membership of the task group was A. Bell, D. Brenan, C. Broad, K. Brown, J. Bull, G. Dimino, J. Emmett, B. Fattorini, A. Faust, B. Harris, U. Henry, E. Mcdermid, O. Morgan, J. Nunn, M. Porter, I. Rodrigues, C. Sleight, and M. Yonge.

Mark Yonge, Chair
Brooks Harris, Vice-Chair
SC-06-01, 1999-06

## Foreword to second edition

This revision of AES31-3 incorporates previous corrigenda, and includes a number of editorial updates and corrections based on experiences of practical implementation. It has also been extended to include support for higher sampling frequencies and simple gain and surround pan automation.

At the time of revision, the writing group included: J. Bull, R. Caine, A. Faust, C. Garcha, B. Harris, U. Henry, J. Palmer, M. Yonge.

Mark Yonge
Chair, SC-02-08

## Foreword to 2019 edition

Various editorial corrections have been made. The order of annexes has been changed and the informative reference section was converted to a bibliography to conform with the IEC recommended format.

Richard Cabot
AES Standards Manager

# AES standard for network and file transfer of audio - Audio-file transfer and exchange - Part 3: Simple project interchange

## 0 Introduction

### 0.1 Rationale for this standard

Time-code character format (TCF) is a modified version of the familiar ASCII time-code display. It supports sample-accurate audio editing while integrating easily with systems and processes requiring conventional time codes. It can carry additional information describing time-base derivation. This generalized specification describes features that may not be applicable in all implementations.

Audio sampling resolution provides sample-accurate positions using a video frame count plus audio sample count scheme. The sample count is appended to the time code together with a sampling frequency indicator. Supported sampling frequencies include the primary audio rates together with the pull-up and pull-down rates required for sampling frequency conversions. Sampling frequency is expressed relative to true seconds.

In addition, TCF provides a means for communicating information about the timing relationships between different source media.

Society of Motion Picture and Television Engineers (SMPTE), European Broadcasting Union (EBU), and film time codes indicate how the HH:MM:SS:FF portion of the time code will be interpreted (30 frames per time base for SMPTE, 25 for EBU, 24 for film).

Time base indicates the actual speed (true hertz) at which the time code and the picture material associated with it is resolved. This has two values, 1 Hz and 1/1,001 Hz.

Video fields and time-code mode indicate a) the video field (field one or two) for National Television Systems Committee (NTSC) or Phase Alternating Line (PAL), and b) the frame-code mode (drop frame, non-drop frame) if the time code is NTSC.

Film framing indicates how film frames are transferred to form the video field sequence.

These enhancements provide an underlying mechanism for a broad range of edit data exchange formats capable of tracking film/video/audio phase relationships and conversions throughout the production process. TCF's similarity to conventional time code means TCF can interchange with conventional time-code formats with little difficulty.

### 0.2 Conventions used in this standard

#### 0.2.1 Decimal points

According to IEC directives, the comma is used in all text to indicate the decimal point. However, in the specified coding, including the examples shown, the full stop is used as in IEC programming language standards.

#### 0.2.2 Data representation

In this standard, all coding and data representations are printed in an equally spaced font.

#### 0.2.3 Non-printing ASCII characters

Non-printing characters are delimited by angle brackets, as in `<CR>` for carriage return.

## 1 Scope

This standard provides a convention for expressing edit data in text form in a manner that enables simple and accurate computer parsing while retaining human readability. It also describes a method for expressing time-code information in character notation. It supports common professional audio sampling frequencies, video frame rates, and film framing. This document addresses the core need of the AES31 series of standards in providing a simple but extensible system for passing audio material between systems.

## 2 Normative references

The following standards contain provisions which, through reference in this text, constitute provisions of this document. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the indicated standards.

ISO/IEC 646:1991, *Information technology — ISO 7-bit coded character set for information interchange*. International Standards Organisation, Geneva, Switzerland.

ISO/IEC 11578, - *Information technology - Open Systems Interconnection - Remote Procedure Call (RPC) [Annex A: Universal Unique Identifier];* International Standards Organisation, Geneva, Switzerland.

ISO 8601, *Data elements and interchange formats - Information interchange - Representation of dates and times*; International Standards Organisation, Geneva, Switzerland.

AES31-2, *AES standard on network and file transfer of audio – Audio-file transfer and exchange – File format for transferring digital audio data between systems of different type and manufacture,* Audio Engineering Society, New York, NY., US.

# 3 Definitions

**3.1**
**audio decision list**
ADL
interchange list format that also forms part of a document conforming to AES31-3

**3.2**
**time base**
nominal second having one of two values, 1,000 s and 1,001 s

> NOTE In a system running at a frame rate of 30 frames per second, exactly one second of real time elapses during the scanning of 30 frames. An example of such a system is an 1125/60 high-definition system. In an NTSC television system running at a vertical field rate of 60/1,001 field/s (approximately 59,94 Hz), one second of NTSC time elapses during the scanning of 60 television fields or 30 television frames, as described in SMPTE 12M, sub-clause 4.1. Because of the difference in vertical scanning rates, the relationship between real time and NTSC time is that 1 s NTSC is equivalent to 1,001 s real time.

**3.3**
**NTSC time**
time base of 1,001 s

**3.4**
**nondrop frame**
uncompensated mode frame numbered 0 through 29 successively with no omissions

**3.5**
**drop frame**
frame compensated using NTSC time counting mode. See 5.1

**3.6**
**ASCII**
7-bit coded character set, per ISO 646, which describes the character encoding used by this standard

**3.7**
**broadcast wave format**
BWF
audio data file format (see AES31-2)

# 4 Edit decision markup language (EDML)

## 4.1 Description

EDML is a very simple markup language designed to accommodate the requirements of edit data exchange. A primary design objective is to maximize platform and transmission compatibility and to provide simple and explicit parsing. It comprises a set of rules for creating ASCII edit data exchange formats and components.

## 4.2 Text specifications and character set

EDML stores and displays data in 7-bit ASCII text form. To help assure compatibility across platforms and transmission links EDML is restricted to a limited character set. Active characters represent the information contents of EDML. White space delimits fields (columns) within records (rows). Carriage return followed by line feed `<CR><LF>` and form feed `<FF>` may be used in their usual manner to accommodate display formatting but are ignored by the EDML parser.

Active character set:

```
abcdefghijklmnopqrstuvwxyz

ABCDEFGHIJKLMNOPQRSTUVWXYZ

0123456789

!"#$%&'()*+,-./:;<=>?@[]^_`{|}~
```

Delimiters:

<space>, <tab>, carriage return <CR>, linefeed <LF>, formfeed <FF>, Esc <ESC>

### 4.3 Keywords

EDML uses special keywords to identify elements.

#### 4.3.1 Sections

A computer system is required to construct and parse text lists. To simplify these tasks and to support modular software development, the list is partitioned into discrete sections, each with a specific purpose. Each section has an explicitly defined start and end using a conventional tag technique.

<A_SECTION> identifies the start of section A_SECTION

</A_SECTION> identifies the end of section A_SECTION

Sections may be nested to form hierarchical structures. Parsing applications that do not recognize a section tag should seek forward to the end of that section in order to skip it and move on to the next. The sequence of sections at any level of a hierarchy is arbitrary.

#### 4.3.2 Entries

Within each section, each EDML compliant format defines keywords, which begin and delimit entries and define the contents of the entries.

The text defining entries and their contents shall not be case dependent.

Entry keywords are composed of a specified keyword surrounded by parentheses.

EXAMPLE:
```
(KEY_WORD)
```

An entry may comprise one or more data values, or fields, corresponding with the data types defined in 4.4.

EXAMPLE 1
```
(SINGLE_VALUE) 0001
```

EXAMPLE 2
```
(RECORD_FIELDS) 0001 00:00:00:00/0000 "Some String"
```

An underscore character (ASCII $5F_{16}$) shall be used in place of any field where valid data are unavailable.

EDML reserves certain entry keywords. See 4.6.

### 4.3.3 Delimiters

Statements, entries, and events within the list are delimited and identified by specific keywords.

Fields within entries shall be delimited by white space. White space is defined as one or more of `<space>`, `<tab>`, `<linefeed>`, `<formfeed>`, or `<carriage return>` in any combination.

White space shall be ignored immediately following a keyword.

Line terminators within entries for print formatting shall comprise the two white space characters `<CR><LF>`.

Embedded entities, such as URIs, shall use their own internally-defined delimiters which may differ from this specification.

### 4.3.4 Comments

Comments are general purpose and distinct from entries using remark keywords (`Rem`) and are not expected to be machine readable. The parser shall be prepared to ignore them.

Comments may be placed in the list using a leading slash-asterisk, `/*` and a terminating asterisk-slash `*/`.

Comments can be placed anywhere in an EDML except within other entries. Comments shall be restricted to the characters defined as part of the EDML character set.

```
/* you can put a comment anywhere except within a record */

/*
you can let it wrap
            if you want, but it shall be terminated with a star-slash */
```

## 4.4 EDML data types

### 4.4.1 Integer numbers

Negative numbers shall be preceded by a minus sign (ASCII $2D_{16}$). Positive numbers may be preceded by an optional plus sign (ASCII $2B_{16}$).

Leading zeros may be used for visual formatting.

### 4.4.2 Real numbers

The EDML compliant format specifies the range of a real number in its field specification.

The decimal point may float.

Negative numbers shall be preceded by a minus sign (ASCII $2D_{16}$). Positive numbers may be preceded by an optional plus sign (ASCII $2B_{16}$).

EXAMPLE 1
`1.12345` provides for a number with six significant figures

EXAMPLE 2
`123.9876` provides for a number with seven significant figures

Leading zeros may be used for visual formatting.

### 4.4.3 Strings

Literal strings shall be enclosed in double quotes.

```
"String"
```

Unless specified otherwise, string lengths are limited to 255 printable characters that comply with 4.2.

Any double-quote character (ASCII $22_{16}$) within a string shall be preceded by an escape character (ASCII $1B_{16}$). For the purposes of documenting this standard, the escape character is shown as <ESC>.

```
"She said<ESC>"Hello<ESC>""
```

NOTE because strings imported from other applications may be of arbitrary size, may include double-quote characters that require an additional escape character, and may include non-printing characters, the total string length could exceed the 255 printable characters specified here. The prudent implementor will take care to accommodate the total number of characters in the string, or implement a suitable truncation policy.

### 4.4.4 Variable-length binary data

Binary data shall be UU encoded for protected transmission as ASCII symbols.

The EDML variable-length binary data type uses a special keyword construction beginning with a defined keyword followed by a data length, in bytes (including UU header), as an EDML number (decimal integer) followed by a single white space followed immediately by the binary data, including any header data required by the encoder.

EXAMPLE
```
(KEYWORD)46 ;/[2=MT!G*,H,'\!@C5/H^&25FGNV'8SA4K58>G#N7H+9
```

To keep the binary data length exact, no white space shall be inserted between the keyword and the data length and precisely one white space character shall be inserted between the data length and the first byte of binary data. This is a stricter requirement than is assumed elsewhere in EDML.

### 4.4.5 Boolean

The EDML Boolean data type consists of two words, TRUE and FALSE, with no surrounding punctuation.

EXAMPLE 1
```
TRUE
```

EXAMPLE 2
```
FALSE
```

### 4.4.6 UUID

The format of the Universal Unique Identifier (UUID) shall comply with ISO/IEC 11578, annex A, using the string representation. Each field of the UUID is treated as an integer and has its value presented as a zero-filled hexadecimal digit string with the most significant digit first. The hexadecimal values a to f inclusive shall be output as lower case characters, and shall be case insensitive on input.

```
00000000-0000-0000-0000-000000000000
```

This example shows the only special case known as a “null UUID” which is clearly not unique. The following is an example of the string representation of a UUID:

```
2fac1234-31f8-11b4-a222-08002b34c003
```

## 4.5 EDML usage conventions

### 4.5.1 Version

Version shall be represented as a string with up to 15 characters including point separators having the format `MJ.MI(.BG)(.ST)(.RV)`, where:

| | | |
|---|---|---|
| `MJ` | Major version number | Required |
| `MI` | Minor release number | Required |
| `BG` | Bug revision (point-release) number (where appropriate) | Optional |
| `ST` | Stage: developmental, alpha, beta, final (where appropriate) | Optional |
| `RV` | Revision of non-released code (where appropriate) | Optional |

EXAMPLE 1
`01.01.00.00.00`

EXAMPLE 2
`01.01`

If data are unavailable, an underscore character shall indicate an empty field.

### 4.5.2 Date, time of day, and zone

Coordinated Universal Time (UTC) is the time scale maintained by the International Bureau of Weights and Measures (BIPM), with assistance from the International Earth Rotation Service (IERS), which forms the basis of a coordinated dissemination of standard frequencies and time signals. Its representation is described in ISO 8601.

#### 4.5.2.1 Date

The date shall be represented as the year, month, and day of the Gregorian calendar. Hyphens (`-`) shall be used as separators within the date expression. All components of the date are required. In the case of unavailable data, the default value shall be the origin of the modified Julian date (MJD), namely, 1858-11-17.

Format: `CCYY-MM-DD`

`CCYY` = 4 characters for century and year — required — shall contain a value 0000 - 9999
`-` = 1 character — required
`MM` = 2 characters for month — required — shall contain a value 1 to 12
`-` = 1 character — required
`DD` = 2 characters for day of month — required — shall contain a value 1 to 31

#### 4.5.2.2 Time of day

The time of day may be appended to the date. If given, it shall be represented as hours, minutes, and seconds. The character `T` shall be used as time designator to indicate the start of the representation of the time of day and shall immediately follow the date. Colons (`:`) shall be used as separators within the time of day expression.

If time is given, all components of the time component shall be present.

Format: `Thh:mm:ss`

`T` = 1 character — required if time given
`hh` (hour) = 2 characters — shall contain a value 00 to 23 if time given
`:` = 1 character — required if time given
`mm` (minute) = 2 characters — shall contain a value 00 to 59 if time given
`:` = 1 character — required if time given
`ss` (second) = 2 characters — shall contain a value 00 to 59 if time given

#### 4.5.2.3 Zone

The relation of local time of day to Coordinated Universal Time (UTC) may be indicated by the zone designator, as shown in annex D.

The local time of day may be expressed directly as UTC. In this case the date and time of day shall be followed immediately, without spaces, by the zone designator `Z`.

The local time of day may be expressed as the difference between local time and UTC. In this case the difference shall be expressed as positive hours and minutes (for example, with a leading plus sign (+)) if the local time is ahead of UTC and as negative hours and minutes (for example, with a leading minus sign (-)) if it is behind UTC. The colon shall be used as separator between hours and minutes.

Format: `Z` or `+hh:mm` or `-hh:mm`

If zone is expressed as UTC:
`Z` = 1 character — required if zone given

If zone is expressed as difference from UTC:
`+` or `-` = 1 character — required if zone given
`hh` = 2 characters — shall contain a value 00 to 23 if zone given
`:` = 1 character — required if zone given
`mm` = 2 characters — shall contain a value 00 to 59 if zone given

> NOTE It is strongly encouraged to use the complete representation, including the zone designator, so the date and local time of day can be normalized to UTC. The difference between local time and UTC is the preferred method.

**EXAMPLE 1:** Date, time of day, and zone (the date, time of day, and zone in Geneva, Greenwich, and New York at the UTC turn of the century):

| | |
|---|---|
| `2000-01-01T00:00:00Z` | (UTC) |
| `2000-01-01T00:00:00+00:00` | (Greenwich) |
| `2000-01-01T01:00:00+01:00` | (Geneva) |
| `1999-12-31T19:00:00-05:00` | (New York — daylight saving time is not in effect) |

**EXAMPLE 2:** Date and local time of day only:

```
1985-03-04T12:00:00
```

**EXAMPLE 3:** Date only:

```
2001-03-01
1858-11-17                    (Default)
```

### 4.5.3 Time code

Time-code strings shall conform to the rules of time-code character format (TCF) to provide sample-accurate position and synchronization information. See clause 5.

## 4.6 Required sections and reserved keywords

EDML reserves several keywords and requires the use of several of them.

### 4.6.1 List-format declaration

Edit data exchange list formats employing EDML shall be contained entirely within a section defined by specifically named keyword tags. This identifies the list-format type and indicates the beginning and end of the list-format instance.

EXAMPLE
```
<ADL>
...
</ADL>
```

### 4.6.2 Header sections

All interchange lists shall contain one or more header sections to define parameters relating to the VERSION, PROJECT, SYSTEM, and SEQUENCE of the particular list.

EXAMPLE
```
<ADL>
       <VERSION>
       </VERSION>
</ADL>
```

### 4.6.3 List contents

There shall be one or more sections containing entries to define the payload contents of the list as defined in that list format. See clause 6.

### 4.7 Column alignment example

It is anticipated that most EDML lists could be displayed or printed with columns aligned where possible for legibility, but this alignment is not a requirement and will be impossible in some situations.

For instance, the CUT event of ADL has eight data fields, and the following three examples are legitimate for the parser and have identical meaning:

**EXAMPLE 1:**
```
(E)0005 (C) 0010 _ 2 2 00:00:00.18/1601 01:00:00.00/0000 01:00:01.08/1601 E
```

**EXAMPLE 2:**
```
(E)0005 (C) 0010 _ 2 2
             00:00:00.18/1601 01:00:00.00/0000 01:00:01.08/1601 E
```

**EXAMPLE 3:**
```
(E)0005
(C)0010

_
2
2
00:00:00.18/1601
01:00:00.00/0000
01:00:01.08/1601 E
```

# 5 Time-code character groups

## 5.0 General

Time-code character format (TCF) expresses sample counts as time-code values in two groups of characters, time-code group and sample group. The numerical relationship between various frame rates and audio sampling frequencies is shown in annex A.

Sample count shall be zero-based such that midnight is represented by the value zero. Sample count then increments at the relevant sampling frequency. This convention is compatible with Broadcast Wave Format (BWF) file implementations.

The time-code group shall be required and provides frame and field accurate video indexing over an indicated 24-hour period. The optional sample group may be appended to the time-code group to provide audio sample resolution.

## 5.1 Time-code group

The time-code group identifies each video frame by a unique and complete address consisting of an hour, minute, second, and frame number. It shall be made up of four two-digit pairs representing hours (`HH`), minutes (`MM`), seconds (`SS`), and frames (`FF`), separated by indicators (`i`), in the form `HHiMMiSSiFF`. The hours, minutes, and seconds follow the ascending progression of a 24-hour clock beginning with 0 hours, 0 minutes, and 0 seconds to 23 hours, 59 minutes, and 59 seconds.

The frames shall be numbered successively from 0 to *n* as indicated by the frame count and time-base indicator (see 5.1.1). The count may be modified by the drop-frame count mode.

> NOTE Because the vertical field rate of an NTSC television signal is 60/1,001 fields per second (approximately 59,94 Hz) (see SMPTE 12M, sub-clause 4.2.2), straightforward counting at 30 frames per second will yield an error of approximately +108 frames (+3,6 s real time) in one hour of running time. To minimize the NTSC time error, the first two frame numbers (00 and 01) shall be omitted from the count at the start of each minute except minutes 00, 10, 20, 30, 40, and 50. When drop-frame compensation is applied to an NTSC television time code, the total error accumulated after one hour is reduced to 3,6 ms. The total error accumulated over a 24-hour period is 86 ms.

The indicators between the `HH MM SS FF` pairs specify frame count and time base; film framing; and video field and time-code type, respectively. They may be modified by the drop-frame counting mode.

The time-code group shall be required.

### 5.1.1 Frame count and time-base indicator

The first separator in the time-code group shall be the frame count and time-base indicator. It specifies the actual frame rate of the picture and the time code by combining two values, the frame count and the time base.

#### 5.1.1.1 Frame count

The frame count specifies how the frames (`FF`) portion of the time-code group is incremented. The count may be modified if the drop-frame count mode is in effect. Table 1 shows the specified frame counts.

**Table 1 — Frame count**

| Frames | Count |
|---|---|
| 30 | 00 to 29 |
| 25 | 00 to 24 |
| 24 | 00 to 23 |

#### 5.1.1.2 Frame-count division

The time base indicates the value of the nominal second into which the frame count is divided.

#### 5.1.1.3 Coding of frame count and time-base indicator

The combination of frame count and time base shall be encoded into a single ASCII character as the frame count and time-base indicator. Table 2 shows the character for each condition followed by the ASCII character's decimal value in parentheses.

**Table 2 — Frame count and time-base indicator coding**

| Frame count | Time base | | |
|---|---|---|---|
| | Unknown | 1,000 | 1,001 |
| 30 | ? (063) | \| (124) | : (058) |
| 25 | ! (033) | . (046) | / (047) |
| 24 | # (035) | = (061) | - (045) |

### 5.1.2 Film frame indicator

The second separator in the time-code group (character position 6) specifies the sequencing of film frames recorded into video.

#### 5.1.2.1 Coding of film frame indicator

NTSC film framing (2–3 pull-down) shall be coded with the film frame in each of the video fields having the sequence `AaBbyCcDdz`. Table 3 shows the coding for each film frame in each video field in the NTSC 2–3 sequence.

**Table 3 — NTSC 2–3 sequence**

| Video field | Video frame | Film | Indicator |
|---|---|---|---|
| 1 | 1 | 1A | A (065) |
| 2 | 1 | 1A | a (097) |
| 3 | 2 | 2B | B (066) |
| 4 | 2 | 2B | b (098) |
| 5 | 3 | 2B | y (121) |
| 6 | 3 | 3C | C (067) |
| 7 | 4 | 3C | c (099) |
| 8 | 4 | 4D | D (068) |
| 9 | 5 | 4D | d (100) |
| 10 | 5 | 4D | z (122) |
| | | | |
| One-to-one | | | Z (090) |
| Unknown | | | X (088) |
| Not applicable | | | Repeat frame count and time-base indicator. |

PAL-to-film framing (the PAL 11–3 sequence) shall be coded using the sequence

`AaBbCcDdEeFfGgHhIiJjKkWwxLlMmNnOoPpQqRrSsTtUuVvYyz`

Table 4 shows the coding for each film frame in each video field in the PAL 11–3 sequence.

One-to-one film framing shall be indicated with an uppercase `Z` (090).

If the time code represents video containing film frames but the film framing is unknown, an uppercase `X` (088) shall be used.

If film-framing information is not applicable, the frame count and time-base indicator shall be repeated in its place.

### 5.1.3 Video field and time-code type indicator

The third separator in the time-code group (character position 9) identifies both the video field and the time-code type. Table 5 shows the video field and time-code type character coding.

**Table 4 — PAL 11–3 sequence**

| Video field | Film frame | Indicator | | Video field | Film frame | Indicator | |
|---|---|---|---|---|---|---|---|
| 1 | 1 | A | (065) | 26 | 13 | L | (076) |
| 2 | 1 | a | (097) | 27 | 13 | l | (108) |
| 3 | 2 | B | (066) | 28 | 14 | M | (077) |
| 4 | 2 | b | (098) | 29 | 14 | m | (109) |
| 5 | 3 | C | (067) | 30 | 15 | N | (078) |
| 6 | 3 | c | (099) | 31 | 15 | n | (110) |
| 7 | 4 | D | (068) | 32 | 16 | O | (079) |
| 8 | 4 | d | (100) | 33 | 16 | o | (111) |
| 9 | 5 | E | (069) | 34 | 17 | P | (080) |
| 10 | 5 | e | (101) | 35 | 17 | p | (112) |
| 11 | 6 | F | (070) | 36 | 18 | Q | (081) |
| 12 | 6 | f | (102) | 37 | 18 | q | (113) |
| 13 | 7 | G | (071) | 38 | 19 | R | (082) |
| 14 | 7 | g | (103) | 39 | 19 | r | (114) |
| 15 | 8 | H | (072) | 40 | 20 | S | (083) |
| 16 | 8 | h | (104) | 41 | 20 | s | (115) |
| 17 | 9 | I | (073) | 42 | 21 | T | (084) |
| 18 | 9 | i | (105) | 43 | 21 | t | (116) |
| 19 | 10 | J | (074) | 44 | 22 | U | (085) |
| 20 | 10 | j | (106) | 45 | 22 | u | (117) |
| 21 | 11 | K | (075) | 46 | 23 | V | (086) |
| 22 | 11 | k | (107) | 47 | 23 | v | (118) |
| 23 | 12 | W | (087) | 48 | 24 | Y | (089) |
| 24 | 12 | w | (119) | 49 | 24 | y | (121) |
| 25 | 12 | x | (120) | 50 | 24 | z | (122) |
| One-to-one | | Z | (090) | | | | |
| Unknown | | X | (088) | | | | |
| Not applicable | | Repeat frame count and time-base indicator. | | | | | |

**Table 5 — Video field and time-code type character coding**

| Counting mode | Video field | |
|---|---|---|
| | **Field 1** | **Field 2** |
| PAL | . (046) | : (058) |
| NTSC non-drop frame | . (046) | : (058) |
| NTSC drop frame | , (044) | ; (059) |

### 5.2 Sample group

Audio sampling resolution shall be provided by appending five characters, the sample group, to the time code. The first character, or separator, of the sample group designates the audio sampling frequency in use. The last four characters record the integer audio sample count position beyond the frame boundary indicated by the time-code group. Together the time-code group and the sample group represent the exact sample position as video frames plus audio sample count.

Annex A shows the exact mathematical relationships between video frames and audio samples.

The use of sample group count shall be optional.

#### 5.2.1 Coding of audio sampling-frequency indicator

The supported sampling frequencies and the character coding for each shall be as shown in Table 6. Rate name is a string that defines a specific manner in which each of the supported sampling frequencies shall be referred. Indicator code character is the ASCII character used to encode each sampling frequency as TCF sampling-frequency indicator. The derivation column shows how each rate is derived and should be considered the strict definition for each sampling frequency. The sampling-frequency column is the sampling frequency shown as a real number to 15 points of precision. It is important to maintain a precise relationship between sampling frequencies derived from different references in order to ensure problem-free interchange.

#### 5.2.2 Audio sample count

The sample count portion of the sample group shall be four numeric characters representing sample count beyond the video frame boundary indicted by the time-code group. The sample count is interpreted as indicated by the sampling-frequency indicator.

**Table 6 — Audio sampling frequencies, indicator coding, and derivations**

| Rate name | Indicator code character | | Derivation | Sampling frequency |
|---|---|---|---|---|
| S48000 | / | (047) | 48000*1 | 48000 |
| S47952 | - | (045) | 48000*1000/1001 | 47952.047952047952048 |
| S48048 | + | (043) | 48000*1001/1000 | 48048 |
| S46080 | < | (060) | 48000*24/25 | 46080 |
| S50000 | > | (062) | 48000*25/24 | 50000 |
| S44100 | \| | (124) | 44100*1 | 44100 |
| S44055 | ~ | (126) | 44100*1000/1001 | 44055.944055944055944 |
| S44144 | ^ | (094) | 4100*1001/1000 | 44144.1 |
| S42336 | [ | (091) | 44100*24/25 | 42336 |
| S45937 | ] | (093) | 44100*25/24 | 45937.5 |
| S32000 | = | (061) | 32000*1 | 32000 |
| S31968 | ‘ | (039) | 32000*1000/1001 | 31968.031968031968032 |
| S32032 | “ | (034) | 32000*1001/1000 | 32032 |
| S30720 | ( | (040) | 32000*24/25 | 30720 |
| S33333 | ) | (041) | 32000*25/24 | 33333.333333333333333 |
| S96000 | * | (042) | 2*48000 * 1 | 96000 |
| S95904 | & | (038) | 2*48000*1000/1001 | 95904.095904095904096 |
| S96096 | # | (035) | 2*48000*1001/1000 | 96096 |
| S92160 | { | (123) | 2*48000*24/25 | 92160 |
| S100000 | } | (125) | 2*48000*25/24 | 100000 |
| S88200 | @ | (064) | 2*44100*1 | 88200 |
| S88111 | ? | (063) | 2*44100*1000/1001 | 88111.888111888111888 |
| S88288 | $ | (036) | 2*44100*1001/1000 | 88288.2 |
| S84672 | ` | (096) | 2*44100*24/25 | 84672 |
| S91875 | ! | (033) | 2*48000*25/24 | 91875 |

# 6 Audio decision list (ADL)

## 6.0 ADL version

Audio decision lists shall indicate their compliance with this revision of the standard by using the version number specified in Annex F.

## 6.1 List structure

This format is based on ASCII text to promote simple and universal construction, communication, and maintenance. The entire ADL shall be contained within <ADL> and </ADL> section keyword tags. All list content shall be contained in subordinate sections defining header information and edit events.

## 6.2 Edit time markers

This format uses an extended form of conventional time-code representation as a means to express sample positions within an audio clip, or edit time line, in a form compatible with the conventional time codes necessary for machine control and other editing systems.

## 6.3 Sections

To simplify text parsing and to support modular software development the list is partitioned into discrete EDML sections, each with a specific set of functions.

Within each section, one or more identifying keywords are each followed by a defined sequence of data fields.

### 6.3.1 Header sections

#### 6.3.1.1 ADL version header

A <VERSION> header section shall contain version information about this ADL. This information identifies the particular list and also verifies the ADL version, creator application, and platform used by the exporting application. This information is necessary to identify any compatibility issues that may arise.

| | | | |
|---|---|---|---|
| <VERSION> | | Required | |
| (ADL_ID) | SMPTE label | Required | Universal ID of ADL format |
| (ADL_UID) | UUID | Required | Unique identifier for this list |
| (VER_ADL_VERSION) | Version type | Required | ADL format version (see 6.0) |
| (VER_CREATOR) | String | Required | Creator application |
| (VER_CRTR) | Version type | Required | Creator version |
| </VERSION> | | | |

#### 6.3.1.2 Project header

The <PROJECT> header section shall contain information relating to the specific project.

| | | | |
|---|---|---|---|
| <PROJECT> | | Required | |
| (PROJ_TITLE) | String | Required | Project title |
| (PROJ_ORIGINATOR) | String | Required | Originating facility |
| (PROJ_CREATE_DATE) | Date | Required | Creation date and time |
| (PROJ_NOTES) | String | Required | Generic comment string |
| (PROJ_CLIENT_DATA) | String | Required | URI for external file |
| </PROJECT> | | | |

### 6.3.1.3 System header

The <SYSTEM> header section can contain optional information about system setup parameters of the originating system.

| | | | |
|---|---|---|---|
| <SYSTEM> | | Optional | |
| (SYS_SRC_OFFSET) | TCF | Optional | Offset all sources, default 0 |
| (SYS_BIT_DEPTH) | integer | Optional | System audio bit depth, default 16 |
| (SYS_AUD_CODEC) | String | Optional | System audio file format, default BWF specified |
| (SYS_XFADE_LEN) | Integer | Optional | System cross-fade length in samples |
| (SYS_GAIN) | Decimal | Optional | System master gain in decibels |
| </SYSTEM> | | | |

### 6.3.1.4 Sequence header

The <SEQUENCE> header section shall provide global information about the ADL contents and status.

| | | | |
|---|---|---|---|
| <SEQUENCE> | | | Required |
| (SEQ_TITLE) | String | Optional | Sequence title |
| (SEQ_DESCRIPT) | String | Optional | Sequence description |
| (SEQ_SAMPLE_RATE) | Integer | Required | Time-line audio sampling-frequency name from Table 6, column one |
| (SEQ_SAMP_RATE_FACTOR) | Integer | Optional | Time-line audio sampling-frequency multiplier: 2, or 4, default 1 |
| (SEQ_FRAME_RATE) | Integer | Required | Time-line video sampling-frequency name; string corresponding to supported TCF frame counts 30, 25, or 24 |
| (SEQ_ADL_LEVEL) | Integer | Required | 1 = ADL level 1 (default) |
| (SEQ_CLEAN) | Boolean | Required | FALSE = sequence may have overlapping events (default). See also 6.3.2.2 - Rule 8 |
| (SEQ_SORT) | Integer | Optional | Sort condition: 0 = ascending entry number (default) |
| (SEQ_MULTICHAN) | Boolean | Optional | FALSE = all events single-channel (default)<br>TRUE = one or more events contain multiple audio channels |
| (SEQ_DEST_START) | TCF | Required | Originating system time-line destination start, a TCF that also encodes the originating system resolve speed, frame and sampling rates, and time-code type |
| </SEQUENCE> | | | |

### 6.3.1.5 Track-list header

This optional section contains a track list; destination tracks are named for operational identification. Track numbers shall be unique and correspond to the destination channels in the event list. Format shall be:

| **Sequence** | **Name** | **Type** | **Status** | **Default** |
|---|---|---|---|---|
| 1 | (Track) | Keyword | Required | |
| 2 | Track# | Integer | Required | |
| 3 | TrackName | String | Optional | "" |

**EXAMPLE**

```
  <TRACKLIST>
      (Track)  9          "Kick"
      (Track)  10         "Snare"
      (Track)  11         "HiHat"
      (Track)  12         "Tom L"
      (Track)  13         "Tom R"
      (Track)  14         ""
  </TRACKLIST>
```

### 6.3.1.6 Source index header

#### 6.3.1.6.0 Source index structure

This required section shall contain a list of the source material used in the project. Each source shall be identified by an incrementing index number. Source types shall be identified by a key letter.

```
<SOURCE_INDEX>.
(Index) 1234 (F) File_Path UID File_In File_Len Descr Code
             (U) URI         UID File_In File_Len Descr Code
             (T) Tapename  Tape_Orig File_Orig T_Chan F_Chan
(Index) 1235 and so on …
</SOURCE_INDEX>
```

#### 6.3.1.6.1 Index

The index number shall be a non-repeating incrementing integer. A source type definition or definitions shall follow the index number. Each source type shall be preceded by an identifying key letter.

| | | | |
|---|---|---|---|
| `(INDEX)` | Keyword | | Required |
| `1234` | Index number | Integer | Required |

File source types can be described using two different forms of locator to represent the physical location of the BWF audio file.

#### 6.3.1.6.2 File path source type

Key letter `(F)` shall identify a file source using an ASCII file path.

| Seq | Name | Type | Status | Content | Default | Notes |
|---|---|---|---|---|---|---|
| 1 | (F) | Key | Required | Key letter | | |
| 2 | *File_Path* | String | Required | ASCII File Path | | |
| 3 | UID | String | Required | Unique identifier | | |
| 4 | File_In | TCF | Optional | Start TC | _ | Equivalent to BWF time stamp of current file |
| 5 | File_Len | TCF | Optional | Length | _ | Sample length of current file |
| 6 | Descr | String | Optional | Text ID | “_” | Same as BWF description field |
| 7 | Code | String | Optional | Usage code | “_” | Can indicate purpose of file:<br>"N" normal audio file<br>"X" cross-fade render file<br>"A" alternate audio file |

File_Path is a customised ASCII-only locator indicating the absolute path to a file. Although it is suitable for simple implementations in which all audio files are stored on local storage, it is recommended that the more flexible URI locator described below is used for all new implementations.

The File_Path string shall take the format:

<IDENTIFIER><DRIVE><DIRECTORY><FILENAME>

where:

<IDENTIFIER> shall be "URL:file://localhost/" to indicate that the file is on local storage.

<DRIVE> shall be an operating system specific drive identifier. This shall be either a drive name (e.g "My_Disk") or a drive letter (e.g "C:")

<DIRECTORY> shall be an operating system specific path consisting of a series of directory names separated by a forward slash (e.g "/Directory_1/Directory_2/")

<FILE> shall be the file name (e.g "Audio_File.wav")

The File_Path string may contain white space and implementations should ensure that all characters between the string's enclosing double quotes (") are processed.

The unique identifier (UID) shall be as defined for the OriginatorReference field in the BWF file. File_In and File_Len values define the material available in the audio file and permit simple inventory checking while the audio material itself is off line. Such an inventory check could parse the ADL to identify what material from which source files will be needed to recreate the edit; these results can then be compared with File_In, File_Len to confirm availability.

Descr carries text from the Description field of the BWF file. It provides the human-readable name to be available while reading the ADL.

Code is a simple code to assist source organization. An X value shall indicate a render file for a cross fade. An A shall indicate a source file that is not used within the ADL but represents an additional recording for subsequent editing flexibility.

The following is an example of a file source identifier using a File_Path string:

```
(F) "URL:file://localhost/D:/Audio Files/Audio 1-01.wav" UKAES000393143a78090954723746998 _ _ _ N
```

NOTE Some legacy applications may only recognise the ASCII file source type represented by key letter ***(F)***. To ensure compatibility with these legacy applications, implementations may optionally save both an ASCII file source type ***(F)*** as well as a full URI file locator ***(U)***. It is recommended that all new implementations use the URI file locator ***(U)*** for locating file sources.

#### 6.3.1.6.3 URI source type

Key letter (U) shall identify a file source using a Unique Resource Identifier.

| | Name | Type | Status | Content | Default | Notes |
|---|---|---|---|---|---|---|
| 1 | (U) | Key | Required | Key letter | | |
| 2 | *URI* | String | Required | File Locator | | |
| 3 | UID | String | Required | Unique identifier | | |
| 4 | File_In | TCF | Optional | Start TC | _ | Equivalent to BWF time stamp of current file |
| 5 | File_Len | TCF | Optional | Length | _ | Sample length of current file |
| 6 | Descr | String | Optional | Text ID | "_" | Same as BWF description field |
| 7 | Code | String | Optional | Usage code | "_" | Can indicate purpose of file:<br>"N" normal audio file<br>"X" cross-fade render file<br>"A" alternate audio file |

The File Locator (URI) shall be a Unique Resource Identifier as specified in Section 9.1. The URI shall be encoded to convert characters that may not be safely transportable across a network (including white space) into their

corresponding escape sequence (for example: a white-space character shall be encoded as `%20`). Details of the characters considered 'safe' for inclusion in a URI are included in the references in Section 9.1.

The URI will typically be preceded by "`file://`" to indicate a file scheme although other schemes may be used for specialised purposes (as specified in the references in Section 9.1).

The unique identifier (`UID`) shall be as defined for the `OriginatorReference` field in the BWF file. `File_In` and `File_Len` values define the material available in the audio file and permit simple inventory checking while the audio material itself is off line. Such an inventory check could parse the ADL to identify what material from which source files will be needed to recreate the edit; these results can then be compared with `File_In`, `File_Len` to confirm availability.

`Descr` carries text from the `Description` field of the BWF file. It provides the human-readable name to be available while reading the ADL.

`Code` is a simple code to assist source organization. An `X` value shall indicate a render file for a cross fade. An `A` shall indicate a source file that is not used within the ADL but represents an additional recording for subsequent editing flexibility.

The following is an example of a file source identifier using a Unique Resource Identifier (URI):

```
(U) "file:// Local%20Drive/Audio%20Files/Audio%201-01.wav" UKAES000393143a78090954723746998 _ _ _ N
```

#### 6.3.1.6.4 Tape source type

Source index entries may include a tape reference to indicate the original source of the associated file where appropriate. Note that variability of machines and synchronizers means that a consistent sample-accurate relationship between the tape and the file is unlikely.

| | | | | |
|---|---|---|---|---|
| `(T)` | Key letter | | Required | |
| `Tapename` | Tape identifier | String | Required | |
| `Tape_Orig` | Start TC | TCF | Required | |
| `File_Orig` | Start TC | TCF | Required | Equivalent to BWF time stamp of the originally transferred file; corresponds to `Tape_Orig` |
| `T_Chan` | Tape channel | Integer array | Required | Default 1 |
| `F_Chan` | File channel | Integer array | Required | Default 1 |

| Name | Type | Status | Content | Default | Notes |
|---|---|---|---|---|---|
| (T) | Key | Required | Key letter | | |
| Tapename | String | Required | Tape identifier | | |
| Tape_Orig | TCF | Required | Start TC | | |
| File_Orig | TCF | Required | Start TC | | Equivalent to BWF time stamp of the originally transferred file; corresponds to Tape_Orig |
| T_Chan | Integer array | Required | Tape channel | 1 | |
| F_Chan | Integer array | Required | File channel | 1 | |

Key letter `(T)` identifies a tape source that shall correspond to the file source identified within the same source index entry.

`Tapename` simply identifies the physical tape to the operator.

`Tape_Orig` shall be a TCF of the first frame of tape material transferred to the original file. `File_Orig` shall be a TCF representing the corresponding file time stamp.

NOTE These values and their relationship will remain constant even when the actual interchanged audio files are truncated or consolidated later in the production process. See figure 4.

T_Chan and F_Chan are two values that indicate track mapping from tape to file during the original digitization in the same way as used in 6.4.2.2. F-Chan shall default to 1 to indicate a mono file. T_Chan shall indicate the corresponding tape channel. This option is capable of multichannel expressions, which are anticipated to become necessary in the future.

### 6.3.2 ADL event section

#### 6.3.2.1 ADL event keyword

The <EVENT_LIST> section of an ADL is the main edit list itself. It contains a list of edit entries with their events, modifiers, and remarks that make up the edited time-line sequence. An ADL may contain no more than one <EVENT_LIST>.

<EVENT_LIST> Required
Event entries
. . .
</EVENT_LIST>

#### 6.3.2.2 ADL event rules

The following rules govern the use of events in the sequence list.

Rule 1 The destination timeline is an abstract representation of presentation time which extends uninterrupted from the lowest to the highest destination position contained in any entries

Rule 2 The destination timeline represents any number of monaural presentation tracks which may be contained in any entries

Rule 3 Each entry describes an event along a single presentation track of the destination timeline.

NOTE: all entry types have an explicit beginning and end time positions. This implies the length of the event.

Rule 4 Any section of the timeline which is not occupied by an entry shall be considered "silent" for computational or presentational purposes. (Figure 1)

NOTE: Silent sections are implied by the absence of any event occupying it. There is no need for a "filler" entry to occupy sections where no material is to be presented. Thus, the timeline can have "holes" in it.

Rule 5 Entry numbers label each event in ascending order along each track of the destination timeline.

Rule 6 Entries may be ordered and numbered in only two ways:

A: In ascending destination timeline position for each `DestIn` regardless of track number. Thus, entries for all tracks are presented interleaved along the destination timeline as they occur. This may be referred to as, "Ordered by timeline position". (Figure 3a)

B: In ascending destination timeline position along each track. Thus, each track is presented as a group of entries in ascending order of `DestIn`. This may be referred to as, "Ordered by track groups". (Figure 3b)

NOTE: in either case, event numbers are always ascending along each destination track.

Rule 7 More than one entry can occupy the same position on the destination timeline.

NOTE: This rule is necessary to support the design of cross fades. See 6.4.4.5, Cross fades.

Rule 8 Where more than one entry occupies the same position on the destination timeline, the highest entry number has priority. (Figure 2)

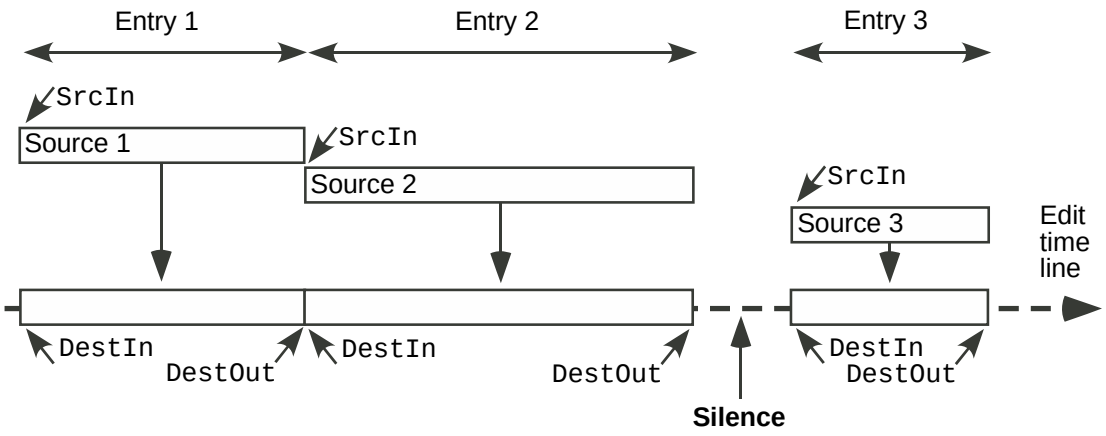



**Figure 1 — Clip assembly, no conflict**

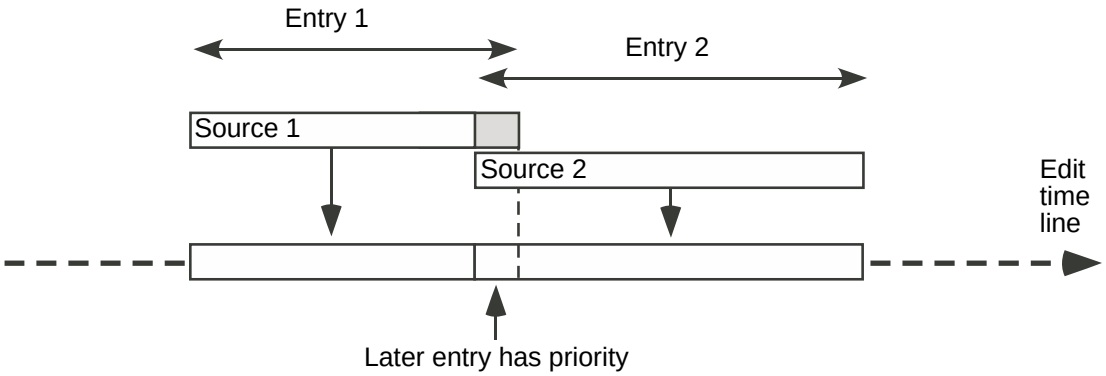



**Figure 2 — Clip assembly, with overlap**

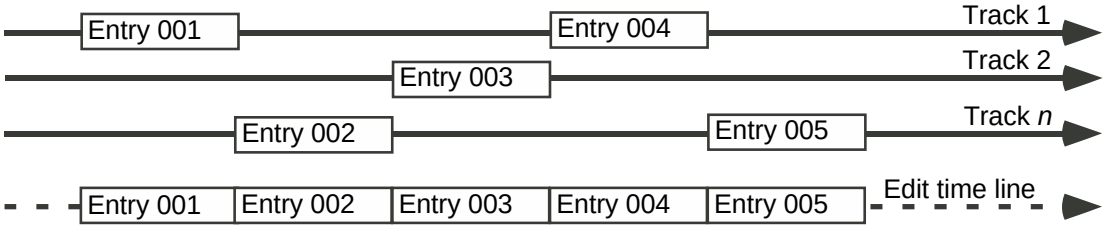



**Figure 3a - Entries ordered by timeline position**

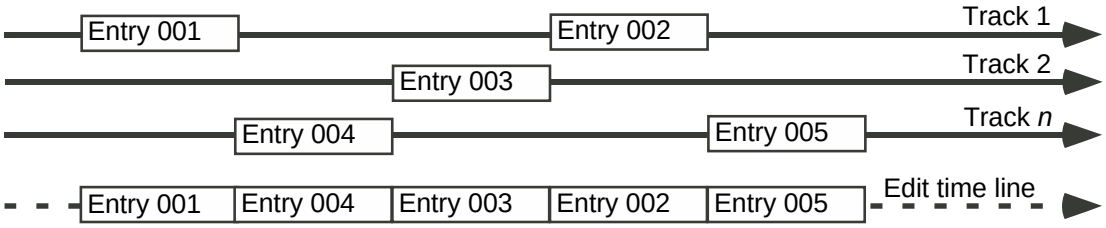



**Figure 3b - Entries ordered by track groups**

## 6.4 ADL event entries

An entry acts to associate various event elements into a single unit identified by an entry number. Statement types that are unrecognized by an application should be ignored.

### 6.4.1 Entries

#### 6.4.1.1 Entry keyword

Each ADL entry shall begin with the keyword `(Entry)` followed by the entry number. The keyword `(Entry)` also serves to delimit the end of any preceding entry.

A second keyword then specifies the primary edit action of the event.

Subsequent keywords identify event modifiers and remarks.

#### 6.4.1.2 Entry number

The entry number data field shall correspond to a positive 32-bit integer. The lowest number shall be 1 or higher; it shall not be zero. Each entry number shall be unique.

#### 6.4.1.3 Event type keyword

Each event type shall be identified by a specific keyword followed by an associated sequence of data fields.

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 1 | `(Entry)` | Keyword | Required | |
| 2 | `EntryNo` | 32-bit integer | Required | |
| 3 | `EventType` | Keyword | Required | (Cut) |

### 6.4.2 Edit event — `Cut`

The sections that follow describe event data field types and how they are used. These data field names exist for the convenience of description and have no formal significance in list parsing .

The fundamental element of the list is the cut edit, illustrated here. Other types of event element can each define a special set of fields.

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 4 | `SrcType` | Key letter | Required | |
| 5 | `SrcIndx` | Integer | Required | |
| 6 | `SrcChan` | Integer | Required | 1 |
| 7 | `DestChan` | Integer | Required | 1 |
| 8 | `SrcIn` | TCF | Required | |
| 9 | `DestIn` | TCF | Required | |
| 10 | `DestOut` | TCF | Required | |
| 11 | `Status` | Key letter | Required | _ (underscore) |

EXAMPLE
A basic `Cut` edit:

```
(Entry) 0010 (Cut) I 1234 1 2 03:00:00;00/0000 01:00:00:00/0000
01:00:10:00/0000 _
```

NOTE The arrangement shown here is for illustration only. The line lengths and field positions are not fixed but simply delimited by white space.

#### 6.4.2.1 Source designation — `SrcIndx`

The first two data fields of an edit event contain the primary source designation (`SrcIndx`). The first field is the letter I ( for "Index"), and the second field is an integer that refers to the specific (Index) source reference in the <SOURCE INDEX> list:

| | |
|---|---|
| `I 01234` | Indirect indexed reference to an `AVIndex` object contained in the `SOURCE_INDEX`. Integer preceded by key letter `I`. |

**Example**: Referencing a source with index 1234:

```
(Entry)0004 (Cut) I 1234 1 2 01.00.00.00/0000 01.00.00.00/0000
01.00.01.00/0000 _
```

#### 6.4.2.2 Channel designation fields — `SrcChan` and `DestChan`

##### 6.4.2.2.1 Single-channel events

Audio channel (track) assignment shall be performed at the event level using two numeric fields. The values for SrcChan and DestChan shall be a positive integer. The lowest number shall be 1 or higher, Negative or zero values are reserved for special purposes.

The `SrcChan` field designates the source channel and defaults to `1` for monaural files and tape sources.

The `DestChan` field designates the channel (track) on the destination time line where the source material should appear. There shall be no upper limit to this integer value.

EXAMPLE 1: Channel 1 from the source file to channel 1 of the destination:

```
(Entry) 0001 (Cut) I 2345 1 1 03:00:00;00/0000
01:00:00:00/0000 01:00:10:00/0000 _
```

EXAMPLE 2: Channel 1 from the source file to channel 24 of the destination time line:

```
(Entry) 0002 (Cut) I 2346 1 24 03:00:00;00/0000
01:00:00:00/0000 01:00:10:00/0000 _
```

##### 6.4.2.2.2 Multichannel events

Contiguous arrays of audio channels, where these exist in the source material, may be defined by two numbers representing the first and last channels of a contiguous set separated by a tilde character (ASCII $7E_{16}$). The same number of channels shall be specified for both source and destination.

> NOTE A single ADL event cannot handle separated sets of channels, that is, a single event cannot make changes to both audio channel 1 and channel 4. Two events are required to accomplish this.

EXAMPLE 1: Channels 1 to 4 from a videotape source to destination channels 9 to 12

```
(Entry)0003 (Cut) I 1231 1~4 9~12 03:00:00;00/0000
01:00:00:00/0000 01:10:00:00/0000 _
```

EXAMPLE 2: Channels 1 to 2, 7, and 12 to 15 from a multitrack audio tape to channels 3 to 4, 23, and 10 to 13 of the destination:

```
(Entry)0001 (Cut) I 1234 1~2 3~4 03:00:00;00/0000
01:00:00:00/0000 01:04:30:00/0000 _
(Entry)0002 (Cut) I 1235 7 23 03:00:00;00/0000
01:00:00:00/0000 01:04:30:00/0000 _
(Entry)0003 (Cut) I 1236 12~15 10~13 03:00:00;00/0000
01:00:00:00/0000 01:04:30:00/0000 _
```

#### 6.4.2.3 Source in — `SrcIn`

This TCF value shall represent the first sample in the selected source, the clip in point.

BWF source files normally carry time-code information in the form of a sample-count-from-midnight time stamp in the file header. The source-in time shall be calculated by adding this time stamp to any additional file offset to produce a value that will remain constant even if the file is subsequently truncated or consolidated (figure 4).

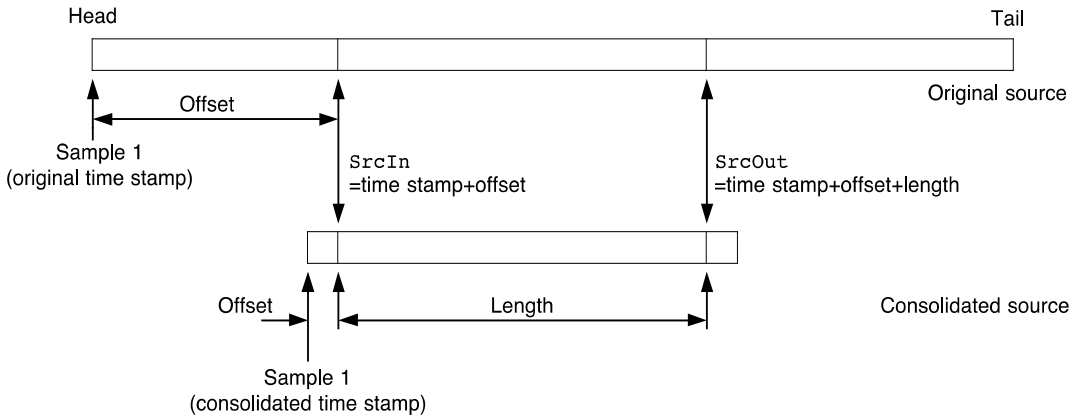



**Figure 4 — Calculating sample positions within files**

Where a time stamp is unavailable, the source-in time shall be calculated simply from the file offset.

#### 6.4.2.4 Destination in — `DestIn`

This TCF value shall represent the start location for this clip in the destination time line, the in point.

#### 6.4.2.5 Destination out — `DestOut`

This TCF value shall represent the end location for this clip in the destination time line, the out point.

#### 6.4.2.6 Status

A status character field allows events in the list to be temporarily marked. This allows, for instance, using an EDL for a partial construction of the project. On reloading this marked up EDL the assembly can continue while ignoring events marked as completed.

The status indicator shall be required.

Status
`R` - Performed (recorded, assembled, conformed)
`E` - Error, bad entry
`D` - Disabled (deferred)
`X` - Marked (generic user mark)
`M` - Mute, do not play
_ - NA (not in use or not applicable) (underscore = ASCII $5F_{16}$) default

### 6.4.3 Edit events, other types

#### 6.4.3.1 Silence

A silence event operates exactly as a cut event with the exception that no source material need be specified.

Field list:

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 3 | (Silence) | Keyword | Required | |
| 4 | DestChan | Integer | Required | 1 |
| 5 | DestIn | TCF | Required | |
| 6 | DestOut | TCF | Required | |
| 7 | Status | Character | Required | |

See 6.4.2 for field definitions.

**EXAMPLE**

```
(Entry) 0004 (Silence) 2 01.00.00.00/0000 01.00.01.00/0000 _
```

#### 6.4.3.2 Auxiliary

Auxiliary events are used for non-synchronized sources: room tone, atmosphere loops, wild audio (microphone), GPIs, etc. Keyword shall be (Aux).

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 3 | (Aux) | Keyword | Required | |
| 4 | DestChan | Integer | Required | 1 |
| 5 | DestIn | TCF | Required | |
| 6 | DestOut | TCF | Required | |
| 7 | Status | Character | Required | |

See 6.4.2 for field definitions.

**EXAMPLE**

```
(Entry) 0004 (Aux) 2 01.00.00.00/0000 01.00.01.00/0000 _
```

#### 6.4.3.3 Video

Video events shall be used to compile a video guide where necessary. A single destination time-line track is supported. Keyword shall be (Vid).

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 3 | (Vid) | Keyword | Required | |
| 4 | SrcType | Key letter | Required | "I" |
| 5 | SrcIndx | Integer | Required | |
| 6 | SrcIn | TCF | Required | |
| 7 | DestIn | TCF | Required | |
| 8 | DestOut | TCF | Required | |
| 7 | Status | Character | Required | |

See 6.4.2 for field definitions.

**EXAMPLE**

```
(Entry) 0007 (Vid) I 6789 01.00.01.00/0000 01.00.00.00/0000 01.00.01.00/0000 _
```

### 6.4.4 Event modifiers

Event modifiers add sets of additional data within an entry, each set preceded by its identifying keyword.

#### 6.4.4.1 Alternate source modifier

Where an alternative source is available for an event, its source reference and equivalent in point may be added as a modifier to an event. Modifier keyword shall be `(Alt)`.

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 3 | `(Alt)` | Keyword | Required | |
| 4 | `SrcType` | Key letter | Required | |
| 5 | `SrcIndx` | Integer | Required | |
| 6 | `SrcChan` | Integer | Required | 1 |
| 7 | `SrcIn` | TCF | Required | |

**EXAMPLE**

```
(Alt) I 2345 1 07:00:00;00/0000
```

#### 6.4.4.2 Transitions

Anti-click fades are performed by each digital audio workstation (DAW) in its own way, and are assumed to be present at the head and tail of every cut. Where two cuts are butted together, there is assumed to be an anti-click cross fade between the two.

Where rendered transitions are applied to the edit time line, no additional fades shall be applied. Finer fade control shall be handled using a transition modifier (6.4.4.3). Because the range of possible variations is potentially infinite, fades that cannot be described in ADL shall be submitted as rendered files by the exporting application.

#### 6.4.4.3 Fade in

A fade-in transition modifier shall be preceded by the keyword `(Infade)`.

The fade-in shall commence at the beginning of the clip specified in the associated edit event.

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 1 | `(Infade)` | Keyword | Required | |
| 2 | `Duration` | TCF | Required | 0 |
| 3 | `Shape` | `LIN` `CURVE` | Required | `LIN` |
| 4 | `Curve_A` | Decimal | Required | _ |
| 5 | `Curve_B` | Decimal | Required | _ |
| 6 | `Curve_C` | Decimal | Required | _ |
| 7 | `SrcType` | Key letter | Optional | |
| 8 | `SrcIndx` | Integer | Optional | |
| 9 | `SrcIn` | TCF | Optional | |

Field definitions are as defined in 6.4.2 except as follows:

a) Duration
The duration, or length, of the fade shall be expressed in TCF format.

b) Shape
Fades shall be specified by the keywords `LIN` or `CURVE`.

With the keyword `LIN`, the fade is defined as an arithmetically linear transition between – 200 dB and unity gain over the duration of the fade, as shown in figure 5.

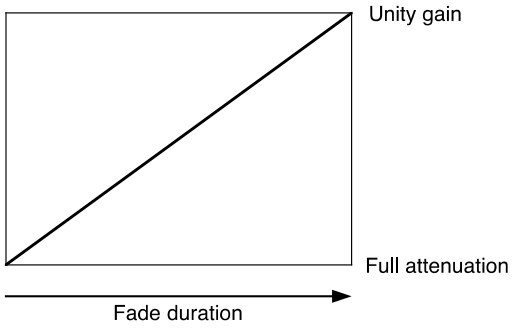



**Figure 5 — Fade shape, linear**

With the keyword `CURVE` the fade is defined as a monotonic sequence of three gain values between 0 dB and – 200 dB, in 0,1-dB steps, as shown in figure 6. Three decimal values shall represent the attenuation of the audio at a quarter, a half and three-quarters of the way through the duration of the fade. The three values shall constitute the three fields following the keyword `CURVE`.

The receiving application shall interpolate these gain points to form a smooth transition. Missing gain values should be calculated by a linear interpolation of surrounding values.

c) Curve A
Attenuation at the quarter duration point.

d) Curve B
Attenuation at the half duration point.

e) Curve C
Attenuation at the three-quarter duration point.

f) Rendered files
The three optional fields in the transition modifier (`SrcType, SrcIndx, SrcIn`) identify a file and an offset pointer to the rendered version of this fade. Field definitions are as defined in 6.4.2 except as shown in this sub-clause.

Where available, receive applications should use the rendered transition files.

**EXAMPLE**

```
(Entry) 0004 (Cut) I 1234 1 2 01.00.00.00/0000 01.00.00.00/0000 01.00.01.00/0000 _
(InFade) 00:00:02:00 LIN I 1235 00.00.00.00/0230
(OutFade) 00:00:02:00 CURVE -4.0 -9.2 -18.9 I 1236 00.00.00.05/6950
```

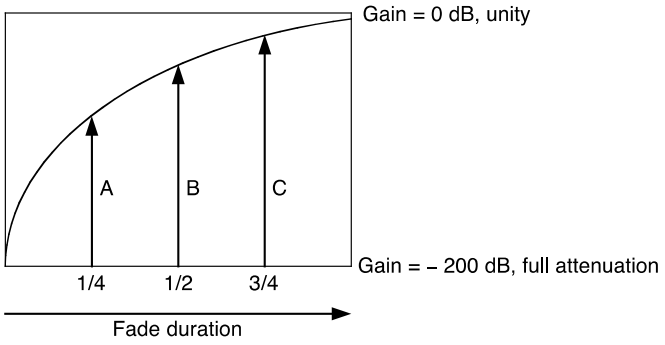



**Figure 6 — Fade shape, curve**

#### 6.4.4.4 Fade out

The fade-out transition modifier is identical to the fade-in with the following exceptions:

a) the fade-out transition modifier shall be preceded by the keyword `(OutFade)`;

b) the fade-out shall conclude at the end of the clip specified in the associated edit event, as shown in figure 7.

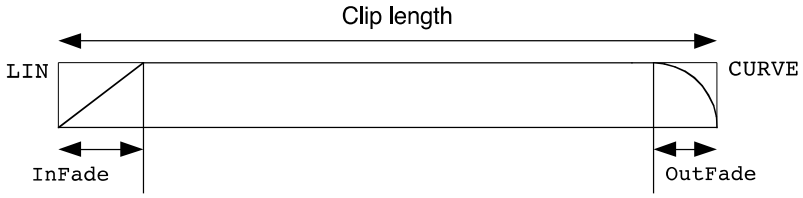



**Figure 7 — Fade positions within a clip**

#### 6.4.4.5 Cross fade

Cross fades describe faded transitions from one audio clip to another, as shown in figure 8.

Cross fades will not be playable on some systems and, for this reason, exporting applications should render them to a dedicated file.

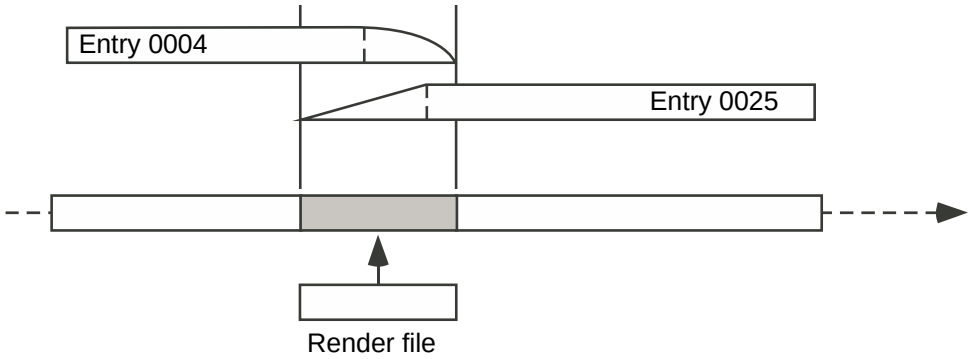



**Figure 8 — Cross fade with rendered file**

An ADL cross fade defines a region of the edit time line where two clips, including any `OutFade` and `InFade` transitions, overlap. It also identifies the `Entry` defining the edit event immediately preceding the current entry on

the relevant destination channels. Where a rendered cross-fade file is not available, the receiving application may reconstruct the cross fade using the information provided in the two edit events.

If the rendered file is not available, and it is not practical to generate a cross fade locally, the cross-fade modifier shall be ignored. The transition effectively reverts to a cut.

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 1 | (Xfade) | Keyword | Required | |
| 2 | PreviousClip | Integer | Required | |
| 3 | DestIn | TCF | Required | |
| 4 | DestOut | TCF | Required | |
| 5 | SrcType | Key letter | Required | |
| 6 | SrcIndx | Integer | Required | |
| 7 | SrcIn | TCF | Required | |

**EXAMPLE**

```
(Entry) 0004 (Cut) I 5325 1 2 05.00.03.21/0052
                                  01.00.00.00/0000 01.00.10.00/0000 _
    (OutFade) 00:00:02:00 CURVE -4.0 -9.2 -18.9 I 3453 00.00.00.05/6950
(Entry) 0025 (Cut) I 1238 1 2 08.30.12.01/5326
                                  01.00.07.00/0000 01.00.22.00/0000 _
    (InFade)  00:00:02:00 LIN _ _ _             I 3454 00.00.00.00/0230
    (Xfade)   0004 01.00.07.00/0000 01.00.10.00/0000
                                                I 3456 00.00.02.19/2500
```

#### 6.4.4.6 Signal modifiers

Signal modifiers control the behavior of the destination channel specified in the associated edit event. Signal modifiers affect the entire duration of an event.

##### 6.4.4.6.1 Gain modifier

The gain modifier specifies a destination channel or channels and defines a gain value for the duration of the associated event for those channels. Units shall be decimal decibels relative to system unity gain. It applies to all channels of the associated entry.

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 1 | (Gain) | Keyword | Required | |
| 2 | Channel | Integer | Optional | _ (underscore) |
| 3 | Gain | Decimal | Required | 0 dB (unity gain) |

**EXAMPLES**

```
(Gain) _     -5.3 /* 5.3 dB attenuation on all associated channels*/

(Gain) 3     +4.5 /* 4.5 dB gain on channel 3*/

(Gain) 1~8  -3   /* 3 dB attenuation on channels 1 to 8*/
```

### 6.4.5 Remarks

Several types of remarks may accompany an entry.

| Sequence | Name | Type | Status | Default |
|---|---|---|---|---|
| 1 | (Rem) | Keyword | Required | |
| 2 | RemType | Text | Required | USER |
| 3 | Remark | String | Optional | _ |

The remark type shall be one of: `NAME, SOURCE, DESC, USER.`

EXAMPLE 1 Edit name remark
A remark to display a name associated with the resulting clip on the edit time line:
```
(Rem) NAME "Door slam"
```

EXAMPLE 2 Source name remark
A remark to identify the name of the source that the event is derived from:
```
(Rem) SOURCE "Slate 104 Take 3"
```

EXAMPLE 3 Event description remark
Description of an event, or comment:
```
(Rem) DESC "This describes what's going on here"
```

EXAMPLE 4 User remark
```
(Rem) USER "A string for user notes not formalized elsewhere"
```

NOTE Entry remarks are distinct from EDML comments (`/* this is a comment */`).

# 7 Edit automation

## 7.0 Introduction

In many situations, automation data has a similar role to edit data to ensure that audio playback on one system is equivalent to playback on another system. Volume automation (often referred to as "gain automation") and mute automation are important parts of audio interoperability. Pan automation is also becoming increasingly important when specifying playback for multi-channel surround audio projects.

Elaborate automation schemes are beyond the scope of workstation interchange intended here and better suited to dedicated mixing console automation systems. Less elaborate approaches to edit automation are both reasonably simple to implement on a wide variety of systems and have been shown to provide acceptable interchange.

This scheme provides a simple method of describing edit automation in a method that can be interpreted by each system using its own capabilities. This is particularly true of pan automation where the method of achieving the spatial placement of audio within a sound field can differ widely between systems.

## 7.1 Description

Volume, pan and mute automation are each described as an EDML section. Each is a list of time points and associated data values along the destination timeline. These lists are data structures whose time-points parallel the time-points of the `<EVENT-LIST>` section of the ADL.

## 7.2 Fader section

### 7.2.1 General

Volume automation is implemented in the form of a fader automation section.

The fader automation section shall be defined by the tag `<FADER_LIST>`.

### 7.2.2 Fader-point entries

Fader-point entries are contained within the Fader section. Each entry represents a fader value at a time-point along the timeline.

Fader-point entries shall be identified by the keyword `(FP)` followed by a sequence of data fields.

| Sequence | Name | Data Type | Status | Default |
|---|---|---|---|---|
| 1 | (FP) | Keyword | Required | |
| 2 | Destination Track | Integer | Required | |
| 3 | DestIN | TCF | Required | |
| 4 | Fader Value | Real | Required | |

### 7.2.3 Fader value

The "Fader" value shall be a real number representing the gain in decibels (relative to the system's unity gain) at the specified timeline position. The value shall be specified to a maximum of 2 decimal places. A maximum gain of +12.00 dB should be used (larger gain values can be used but these may not be supported on all systems).

### 7.2.4 Fader default values

The fader-point value at the beginning of any timeline destination track shall be 0 dB. This default value may be overridden by applying an explicit fader-point coincident with the beginning of the track.

The last fader-point value specified on a destination track shall be assumed to continue without change until the end of the timeline on this track.

### 7.2.5 Sorting

Fader points should be sorted into sequential track order, and into sequential time order within each track.

**Example:**

```
<FADER_LIST>
(FP) 1 00.00.00.00|0000 1.00
(FP) 1 00.00.06.01|1476 -2.00
(FP) 1 00.00.10.03|0000 -4.00
(FP) 2 00.00.06.01|1476 -2.00
(FP) 2 00.00.09.10|0000 3.00
(FP) 2 00.00.10.03|0000 0.00
(FP) 3 00.00.10.03|0000 2.00
</FADER_LIST>
```

### 7.2.6 Interpretation of fader values

The playback system shall apply an arithmetically linear transition between each adjacent fader value on a destination track:

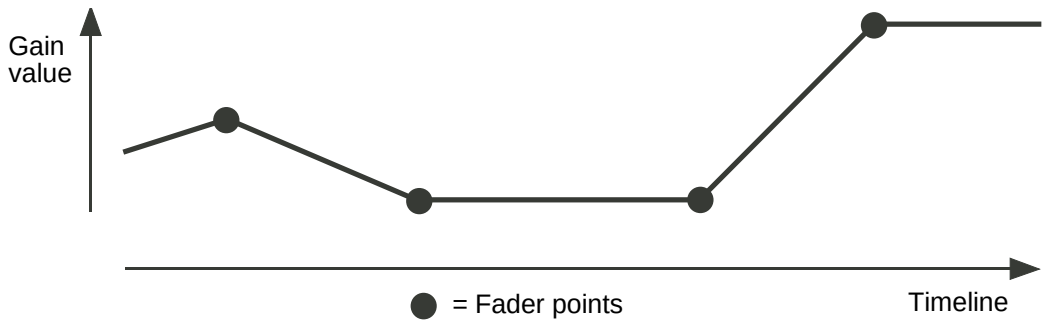



**Figure 9 - Fader transitions**

If it is required to produce an immediate level change at a specified timeline position, this shall be specified by using two fader point values located at least one sample apart on the timeline.

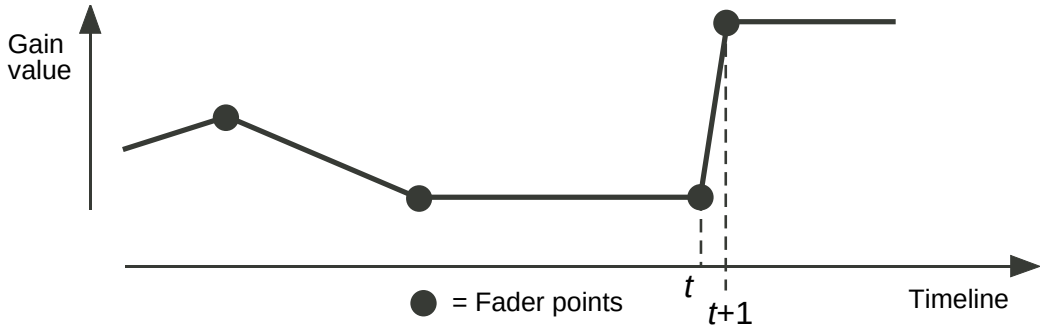



**Figure 10 - Immediate gain change**

### 7.3 Pan section

#### 7.3.1 General

The pan automation section shall be defined by the tag <PAN_LIST>.

#### 7.3.2 Pan-point entries

Pan-point entries are contained within the Pan Section list. Each entry represents a pan value at a time-point along the timeline.

Pan-point entries shall be identified by the keyword (PP) followed by a sequence of data fields.

| Sequence | Name | Data Type | Status | Default |
|---|---|---|---|---|
| 1 | (PP) | Keyword | Required | |
| 2 | DestinationTrack | Integer | Required | |
| 3 | DestIN | TCF | Required | |
| 4 | LeftRightPosition | Real | Required | 0 (centre) |
| 5 | FrontRearPosition | Real | Optional | 0 (front) |

#### 7.3.3 Pan values

Each pan point entry identifies a position in space at which audio should be reproduced. The technique for achieving this placement is an implementation issue and is not specified here. This allows the pan position to be specified independently of system specifics such as the number of speakers or pan curves.

The reproduced sound pressure level shall remain independent of pan value.

For panning across a single left-right axis (such as in the case of stereo speakers), the position is specified by a linear displacement to the left or right. Positive values shall indicate a displacement to the right (where +100.0 is fully right); negative values shall indicate a displacement to the left (where –100.0 is fully left). A default value of 0.0 shall indicate that the audio is panned to the center:

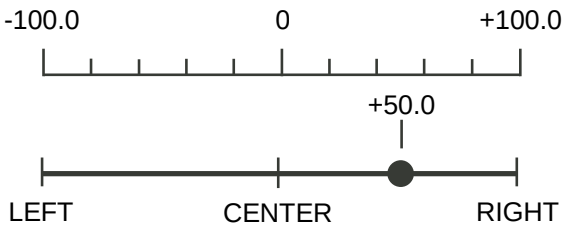



**Figure 11 - Left to Right pan coordinates**

The LeftRightPosition pan value shall be specified to a maximum of 1 decimal place.

An additional parameter (FrontRearPosition) may be used to specify front-to-rear panning in a two-dimensional space. Front-to-rear position shall be specified by linear displacement from the front (0.0) to the rear (200.0).

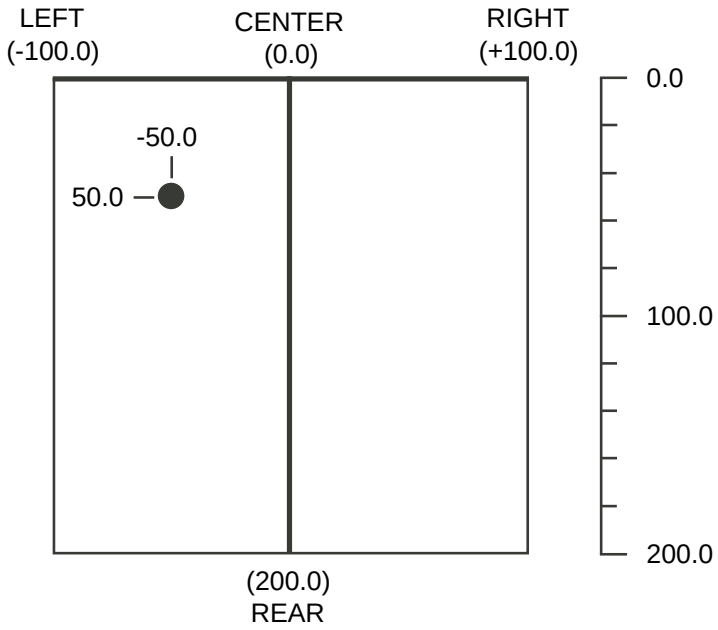



**Figure 12 - Surround pan coordinates**

The FrontRearPosition pan value shall be specified to a maximum of 1 decimal place.

NOTE: There are no valid negative values of FrontRearPosition

**Table 7: Examples of pan values**

| Position | Pan values | |
|---|---|---|
| | Left to right | Front to back |
| Centre | 0 | 0 |
| Left | -100 | 0 |
| Right | 100 | 0 |
| Rear, centre | 0 | 200 |
| Rear, left | -100 | 200 |
| Rear, right | 100 | 200 |
| Half-left, front | -50 | 0 |

### 7.3.4 Pan defaults

The pan value shall default to center and front in the sound field at the beginning of any timeline destination track. This default value may be overridden by applying an explicit panpoint coincident with the beginning of the track.

### 7.3.5 Sorting

Pan points should be sorted into sequential track order, and into sequential time order within each track.

**Example:**

```
<PAN_LIST>
(PP) 1 01.00.33.01|0000 0.0
(PP) 1 01.00.33.02|0000 100.0
(PP) 1 01.00.35.00|0000 100.0
(PP) 1 01.00.38.03|0000 -50.0
(PP) 1 01.00.52.00|0000 -50.0
(PP) 1 01.00.55.03|0000 0.0
</PAN_LIST>
```

NOTE: The effect of these pan points is illustrated in figure 13

### 7.3.6 Interpretation of pan points

The playback system shall apply an arithmetically linear transition between each adjacent pan point values defined on a destination track (figure 13).

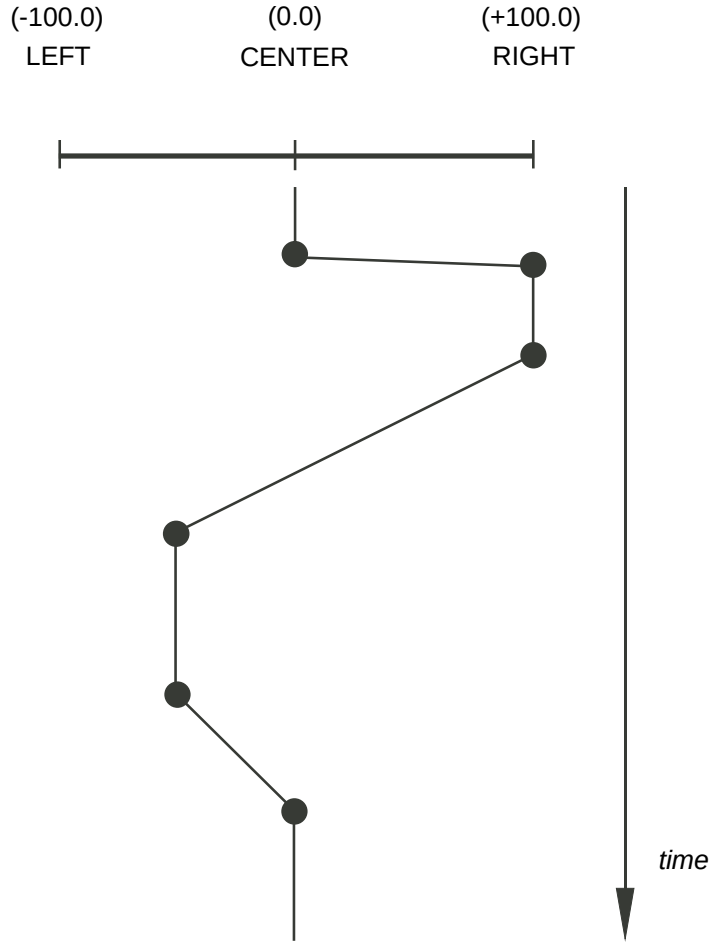



**Fig 13: Dynamic pan example, 2-channel pan, track 1**

## 7.4 Mute section

### 7.4.1 General

The mute automation section shall be defined by the tag <MUTE_LIST>.

### 7.4.2 Mute-point entries

Mute-point entries are contained within the Mute Section list. Each entry represents a mute value at a time-point along the timeline

Mute-point entries shall be identified by the keyword (MP) followed by a sequence of data fields.

| Sequence | Name | Data Type | Status | Default |
|---|---|---|---|---|
| 1 | (MP) | Keyword | Required | |
| 2 | DestinationTrack | Integer | Required | |
| 3 | DestIN | TCF | Required | |
| 4 | Mute | Key letter | Required | |

### 7.4.3 Mute values

Mute values shall be represented by the key letters:

M = audio muted,
U = audio unmuted

### 7.4.4 Mute defaults

By default, audio shall be unmuted at the beginning of any timeline destination track. This default value may be overridden by applying an explicit mute-point coincident with the beginning of the track.

### 7.4.5 Sorting

Mute points should be sorted into sequential track order, and into sequential time order within each track.

**Example:**

```
<MUTE_LIST>
(MP) 1 00.00.06.01|1476 M
(MP) 1 00.00.10.03|0000 U
(MP) 3 00.00.06.01|1476 M
(MP) 5 00.00.10.03|0000 U
</MUTE_LIST>
```

### 7.4.6 Interpretation of mute points

Implementation of mute points is system dependent and is not addressed by this standard.

> Note - Some systems may 'click' if a mute is applied during active audio. Typically users apply mutes at very quiet places on a track to suppress spurious noise or unwanted audio. It is appropriate that the user, not the system, make this selection.

## 7.5 Markers section

### 7.5.1 General

The capacity to mark a track, or a region within a track, is useful for audio exchanged projects and, although these marks are not intended to control audio parameters directly, they support the same objectives as automation. They enable the originating editor to communicate specific times on specific tracks as a guide to future events or processes.

The markers section shall be defined by the tag `<MARKER_LIST>`.

### 7.5.2 Marker-point entries

Marker-point entries are contained within the markers section. Each entry represents a mark at a specified point along the destination timeline of one or all tracks.

Marker-point entries shall be identified by a keyword followed by a sequence of data fields. The keywords are listed in table 8.

**Table 8: Marker keywords**

| Keyword | Function |
|---|---|
| (MK) | Position or region identifier |
| (MK-PQ-START) | CD PQ data; start |
| (MK-PQ-END) | CD PQ data; end |
| (MK-PQ-INDEX) | CD PQ data; index |

| Sequence | Name | Data Type | Status | Default |
|---|---|---|---|---|
| 1 | Keyword (see Table 8) | Keyword | Required | |
| 2 | DestinationTrack | Integer or range | Required | |
| 3 | DestIN | TCF | Required | |
| 4 | DestOUT | TCF | Optional | _ |
| 5 | MarkerName | String | Optional | _ |

### 7.5.3 Mark destination track

#### 7.5.3.1 Single track markers

Each marker shall be identified with a specific track (channel) on the destination timeline according to the value of DestinationTrack which shall be positive and non-zero.

#### 7.5.3.2 Global track markers

The special case of DestinationTrack = `0` shall apply to all available destination tracks (channels) as a global marker.

#### 7.5.3.3 Multiple track markers

Groups of contiguously-numbered tracks may be identified by separating the first and last tracks with a tilda character, ~, (ASCII $2E_{16}$).

Example 1: `1~6` specifies a marker associated with tracks 1 to 6, inclusive
Example 2: `7~8` specifies a marker associated with tracks 7 and 8

> NOTE A single marker point cannot handle separated channels. Markers for non-contiguous tracks will require multiple marker point entries.

### 7.5.4 Mark region

A DestIn parameter on its own shall indicate a single point on the destination timeline. Where a DestOut parameter is also provided, the two points shall indicate a marked time region, or duration. DestOut shall be later in time than DestIn. Where a marker type does not require a DestOut parameter, an underscore character shall be used as a default.

### 7.5.5 Marker name

The MarkerName field may carry a text string for the mark or region.

### 7.5.6 Sorting

Mark-point entries should be sorted into sequential track order, and into sequential time order within each track.

**Example:**

```
<MARKER_LIST>
(MK) 1 00.00.06.01|1476 _ "a simple mark, track 1"
(MK) 5 00.00.10.03|0000 00.01.05.00|0000 "a defined region, track 5"
(MK) 1~8 00.00.10.03|0000 00.00.20.03|0000 "region, 10 s, 8 tracks"
(MK) 0 00.01.00.00|0000 _ "Start of show, all tracks globally"
</MARKER_LIST>
```

### 7.5.7 Interpretation of mark points

Implementation of mark points is system dependent and is not addressed by this standard.

## 8 Reference lists

Reference lists provide an optional mechanism to embed multiple guide track sources within the ADL.

Reference lists (`<REF_LIST>`) contain some number of subordinate reference sections, each of which may be a complete edit list. Each `REF` section may contain `PROJECT`, `SYSTEM`, and `SEQUENCE` header sections and an `EVENT_LIST`. The structure of a reference `EVENT_LIST` is identical to the main `ADL EVENT_LIST`.

```
<REF_LIST>                                  Optional
    <REF>
        <VERSION>                           Required
            Contains version & ID
        </VERSION>
        <REFERENCE>                         Optional
            Contains reference source information
        </REFERENCE>
        <SYSTEM>                            Optional
            Contains system parameters
        </SYSTEM>
        <SEQUENCE>                          Optional
            Contains sequence contents and condition
        </SEQUENCE>
        <EVENT_LIST>                        Required
            Contains reference events
        </EVENT_LIST>
    </REF>
</REF_LIST>
```

# 9 Assignment of coding

## 9.1 Resource locators, identifiers, and formats

The coding of resource locators,, identifiers, and formats shall conform to recognized industry practice as determined by the AESSC according to its rules. This determination is shown as of the printing date of this standard in annex E which shall be published and kept current on the AESSC Web site databases page.

## 9.2 Reserved locations

Application may be made by any directly and materially affected party for assignment of reserved locations by mail to the publisher, Standards Secretariat, Audio Engineering Society, 60 East 42nd St., New York, NY 10165, or by submittal via the AESSC Web site databases page. The secretariat shall place submittals in a Web area for review by the public and the AESSC for three months. If no objection is received within those three months, the data in the submittal shall be placed in the standard as appropriate. All applications shall be reviewed by the AESSC according to its rules and procedures.

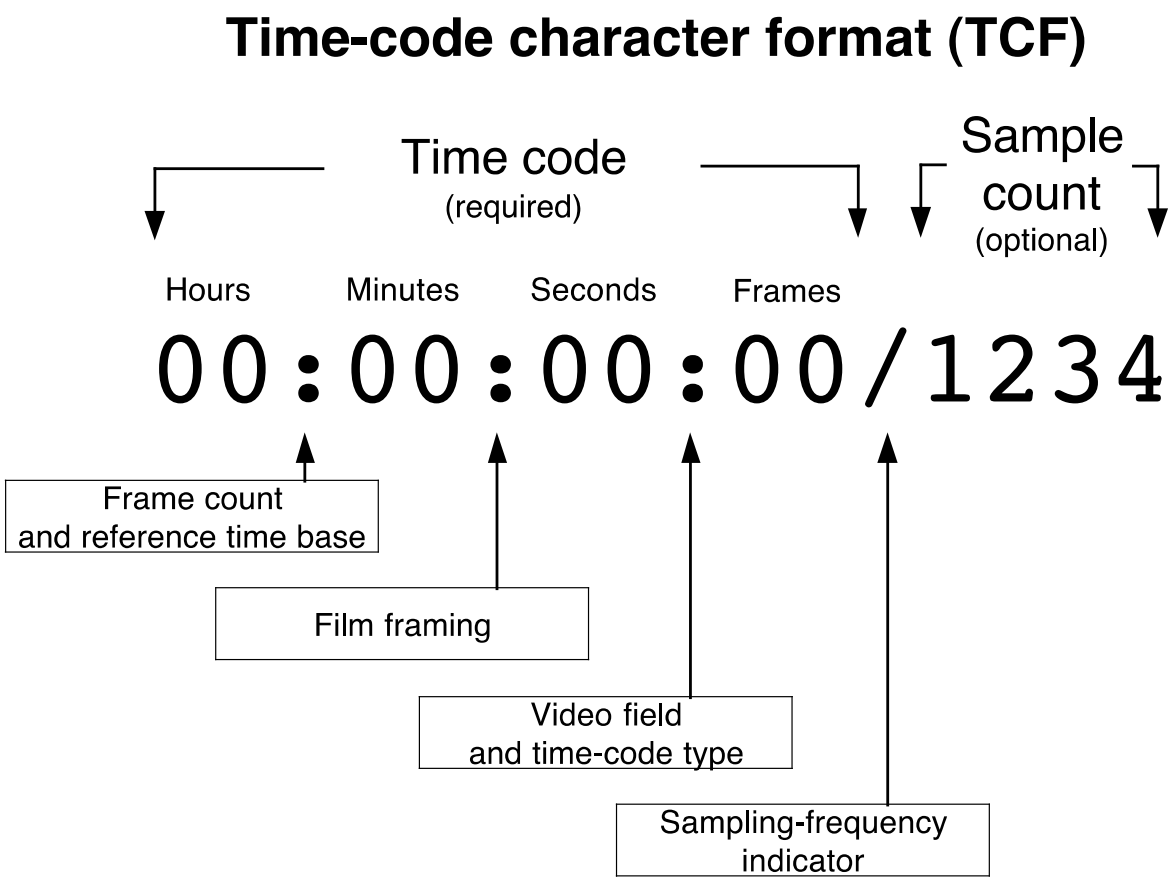
# Annex A (Normative) Video frame rate to sampling-frequency ratios

This table specifies the ratio between video frame rates and audio sampling frequencies for each of the supported sampling frequencies and the formula used to derive them. The ratio is the lowest common factor of the result shown at the far right.

| **Frequency** | **Ratio** | **Derivation** |
|---|---|---|
| ====== 32 NTSC ====== | | |
| 32K: | 15/ 1601 | (30000/1001) * (1/32000) == (30000/32032000) |
| 32-: | 3/ 3200 | (30000/1001) * (1/(32000 * 1000/1001)) == (30000/32000000) |
| 32+: | 1875/ 2004002 | (30000/1001) * (1/(32000 * 1001/1000)) == (30000/32064032) |
| 32<: | 125/ 128128 | (30000/1001) * (1/(32000 * 24/25)) == (30000/30750720) |
| 32>: | 9/ 10010 | (30000/1001) * (1/(32000 * 25/24)) == (30000/1001000000) |
| ====== 44 NTSC ====== | | |
| 44K: | 100/ 147147 | (30000/1001) * (1/44100) == (30000/44144100) |
| 44-: | 1/ 1470 | (300000/10010) * (1/(44100 * 1000/1001)) == (30000/44100000) |
| 44+: | 100000/ 147294147 | (300000/10010) * (1/(44100 * 1001/1000)) == (30000/441882441) |
| 44<: | 625/ 882882 | (300000/10010) * (1/(44100 * 24/25)) == (30000/42378336) |
| 44>: | 96/ 147147 | (300000/10010) * (1/(44100 * 25/24)) == (30000/459834375) |
| ====== 48 NTSC ====== | | |
| 48K: | 5/ 8008 | (30000/1001) * (1/48000) == (30000/48048000) |
| 48-: | 1/ 1600 | (30000/1001) * (1/(48000 * 1000/1001)) == (30000/48000000) |
| 48+: | 625/ 1002001 | (30000/1001) * (1/(48000 * 1001/1000)) == (30000/48096048) |
| 48<: | 125/ 192192 | (30000/1001) * (1/(48000 * 24/25)) == (30000/46126080) |
| 48>: | 3/ 5005 == | (30000/1001) * (1/(48000 * 25/24)) == (30000/50050000) |
| ====== 88 NTSC — same as 44K ====== | | |
| ====== 96 NTSC — same as 48K ====== | | |
| ====== 32 PAL ====== | | |
| 32K: | 1/ 1280 | (25 / 1) * (1 / 32000) == (25 / 32000) |
| 32-: | 1001/ 1280000 | (25025 / 1001) * (1 / (32000 * 1000/1001)) == (25025 / 32000000) |
| 32+: | 25/ 32032 | (25 / 1) * (1 / (32000 * 1001/1000)) == (25 / 32032) |
| 32<: | 5/ 6144 | (25 / 1) * (1 / (32000 * 24 / 25)) == (25 / 30720) |
| 32>: | 3/ 4000 | ((25*150)/(1*150)) * (1/(32000 * 25/24)) == (3750/5000000) |
| ====== 44 PAL ====== | | |
| 44K: | 1/ 1764 | (25/1) * (1/44100) == (25/44100) |
| 44-: | 143/ 252000 | ((25*1001)/(1*1001)) * (1/(44100*1000/1001)) == (25025/44100000) |
| 44+: | 250/ 441441 | ((25*10010)/(1*10010))*(1/(44100*1001/1000)) == (250250/441882441) |
| 44<: | 25/ 42336 | (25/1) * (1/(44100 * 24/25)) == (25/42336) |
| 44>: | 2/ 3675 | ((25*10010)/(1*10010))*(1/(44100 * 25/24)) == (250250/459834375) |
| ====== 48 PAL ====== | | |
| 48K: | 1/ 1920 | (25/1) * (1/48000) == (25/48000) |
| 48-: | 1001/ 1920000 | ((25*1001)/(1*1001)) * (1/(48000*1000/1001)) == (25025/48000000) |
| 48+: | 25/ 48048 | (25/1) * (1/(48000 * 1001/1000)) == (25/48048) |
| 48<: | 5/ 9216 | (25/1) * (1/(48000 * 24/25)) == (25/46080) |
| 48>: | 1/ 2000 | (25/1) * (1/(48000 * 25/24)) == (25/50000) |
| ====== 88 PAL — same as 44K ====== | | |
| ====== 96 PAL — same as 48K ====== | | |

The ratios in this table are used for converting between video frames and audio samples, as is required by the design of TCF. For example, the ratio 5 / 8008 describes the exact relationship of 48kHz audio sampling to NTSC video. This may be used to convert an audio sample number to the nearest video frame and the sample count. Remember that sample numbers are zero-based such that midnight is represented by the number zero.

## Annex B (Informative) Time-code character format



## Annex C (Informative) Section keyword summary

Summary of EDML keywords defined by ADL for sections:

```
<ADL>                                                   Required
        <VERSION>                                       Required
                Contains version information
        </VERSION>
        <PROJECT>                                       Optional
                Contains project information
        </PROJECT>
        <SYSTEM>                                        Required
                Contains system parameters
        </SYSTEM>
        <SEQUENCE>                                      Required
                Contains sequence contents
                and condition
        </SEQUENCE>
        <TRACKLIST>                                     Optional
                Contains track-list information
        </TRACKLIST>
    <EVENT_LIST>                                        Required
                Contains ADL events, modifiers, and remarks
    </EVENT_LIST>
    <FADER_LIST>                                        Optional
          Contains fader values
    </FADER_LIST>
    <PAN_LIST>                                          Optional
          Contains pan values
    </PAN LIST>
    <MUTE_LIST>                                         Optional
          Contains mute values
    </MUTE_LIST>
    <MARKER_LIST>                                       Optional
                Contains marker values
    </MARKER_LIST>
    <REF_LIST>                                          Optional
          <REF>
                Contains reference events
          </REF>
    </REF_LIST>
</ADL>
```

## Annex D (Informative) Zone differences from UTC for some locations

| Hours | Standard time | Daylight saving | Hours | Standard time | Daylight saving |
|---|---|---|---|---|---|
| UTC | Greenwich | | UTC-00:30 | | |
| UTC-01:00 | Azores | | UTC-01:30 | | |
| UTC-02:00 | Mid-Atlantic | | UTC-02:30 | | Newfoundland |
| UTC-03:00 | Buenos Aires | Halifax | UTC-03:30 | Newfoundland | |
| UTC-04:00 | Halifax | New York | UTC-04:30 | | |
| UTC-05:00 | New York | Chicago | UTC-05:30 | | |
| UTC-06:00 | Chicago | Denver | UTC-06:30 | | |
| UTC-07:00 | Denver | Los Angeles | UTC-07:30 | | |
| UTC-08:00 | Los Angeles | | UTC-08:30 | | |
| UTC-09:00 | Alaska | | UTC-09:30 | Marquesas Islands | |
| UTC-10:00 | Hawaii | | UTC-10:30 | | |
| UTC-11:00 | Midway Islands | | UTC-11:30 | | |
| UTC-12:00 | Kwaialein | | UTC+11:30 | Norfolk Island | |
| UTC+13:00 | | New Zealand | UTC+10:30 | Lord Howe Island | |
| UTC+12:00 | New Zealand | | UTC+09:30 | Darwin | |
| UTC+11:00 | Solomon Islands | | UTC+08:30 | | |
| UTC+10:00 | Guam | | UTC+07:30 | | |
| UTC+09:00 | Tokyo | | UTC+06:30 | Rangoon | |
| UTC+08:00 | Beijing | | UTC+05:30 | Bombay | |
| UTC+07:00 | Bangkok | | UTC+04:30 | Kabul | |
| UTC+06:00 | Dhaka | | UTC+03:30 | Tehran | |
| UTC+05:00 | Islamabad | | UTC+02:30 | | |
| UTC+04:00 | Abu Dhabi | | UTC+01:30 | | |
| UTC+03:00 | Moscow | | UTC+00:30 | | |
| UTC+02:00 | Eastern Europe | | UTC+12:45 | Chatham Islands | |
| UTC+01:00 | Central Europe | | | | |

## Annex E (Normative) Currently approved resource locator, identifier, and format references

IETF RFC2396 *Uniform Resource Identifiers (URI).* Internet Engineering Task Force.

# Annex F (Normative) ADL Version

The ADL version header requires a valid version number for the ADL structure (see 6.3.1.1 and 4.5.1).

Audio decision lists shall indicate their compliance with this 2008 revision of the standard by using the major and minor version numbers specified below:

```
01.02
```

## Annex G Bibliography

**AES5-2003**: *AES recommended practice for professional digital audio — Preferred sampling frequencies for applications employing pulse-code modulation,* Audio Engineering Society, New York, NY., US.

**ANSI/SMPTE 12M-1995**, *Television, Audio and Film — Time and Control Code for Video and Audio Tape for 525 Line/60 Field Television Systems*. New York, NY, US: American National Standards Institute.

**ANSI/SMPTE 170M-1994**, *Television — Composite Analog Video Signal — NTSC for Studio Applications*. New York, NY, US: American National Standards Institute.

**ANSI/SMPTE 258M-1993**, *Television — Transfer of Edit Decision Lists*. New York, NY, US: American National Standards Institute.

**ANSI/SMPTE 262M-1995**, *Television, Audio and Film — Binary Groups of Time and Control Codes, Storage and Transmission of Data.* New York, NY, US: American National Standards Institute.

**ANSI X3.4-1996 (R1992)**, *7-bit American National Standard Code for Information Interchange (7-bit ASCII).* New York, NY, US: American National Standards Institute.

**EBU Tech 3285**, *Specification of the Broadcast Wave Format.* European Broadcasting Union, Geneva, Switzerland.

Proposed SMPTE *Engineering Guideline for Television — Conversion of SMPTE 12M Time Code to MPEG-2 PCR Time Base Conversion of Audio Samples to Video Frames.* New York, NY, US: Society of Motion Picture and Television Engineers, Revised 1998-05-08.

**ISO/IEC 2022**:1994, *Information technology — Character code structure and extension techniques.* Geneva, CH: International Electrotechnical Commission.

**ISO/IEC 9899** (1990-12), *Programming languages — C.* Geneva, CH: International Electrotechnical Commission.

**ITU-R BT.470-6**, *Conventional television systems.* Geneva, CH: International Telecommunications Union, Annex.

**ITU-R TF.460-5**, *Standard-frequency and time-signal emissions.* Geneva, CH: International Telecommunications Union.

**RFC1521**, *MIME (Multipurpose Internet Mail Extensions).* Internet Engineering Task Force.

**Open Group DCE1.1**, *Remote Procedure Call*, Open Group, 1997. Appendix A discusses the UUID structure. http://www.opengroup.org/